

Notas Criptografia - Semestre VI

Abraham Nuñez

April 15, 2026

Contents

1	Introducción	2
2	Información	2
2.1	Prerrequisitos:	2
2.2	Evaluación:	3
2.3	Herramientas:	3
3	Temario	3
3.1	Descripción	3
3.2	Unidad 1	3
3.3	Unidad 2	3
3.4	Unidad 3	4
3.5	Unidad 4	4
4	Introducción a la criptografia	4
4.1	STRIDE	4
4.2	Origen de la criptografia	5
4.3	Criptografia	5
4.3.1	Herramientas	6
4.3.2	Cifrado Cesar	6
4.3.3	Tres protecciones principales de la criptografia	7
4.3.4	AES	7
4.3.5	YANAC	7
4.3.6	Codificar	8
4.3.7	Cifrar	8
4.3.8	Ofuscar	8
4.3.9	Hashear	8
4.3.10	Cifrado cesar	8

4.3.11	Aritmetica Modular	8
4.3.12	Binario	9
4.4	Hashing	9
4.4.1	Cifrado vs Hashing	9
4.4.2	DIGESTS	10
4.4.3	hashlib	10
4.4.4	Funciones de hash	10
4.4.5	Hashing	11
4.4.6	Seguridad en funciones de hash	13
4.5	Cifrado simetrico	14
4.5.1	AES	14
4.5.2	Seguridad en cifrado simetrico	15
4.6	Cifrado asimetrico	15
4.6.1	Integridad de mensajes, firmas y certificados	16

1 Introducción

- Profesor: Hector Xavier Limón Riaño
- Materia: Criptografia
- Contacto: xavier120@hotmail.com
- Carrera: Ciberseguridad e Infraestructura de Computo

2 Información

2.1 Prerrequisitos:

- Estructuras de datos (Python)
- Matematicas para la ciberseguridad
- Ciberseguridad
- Manipulacion de archivos de texto y binarios
- Manipulacion de cadenas de texto y binarios
- Manipulacion de datos binarios
- Sockets TCP

2.2 Evaluación:

- Exámenes: 60%
- Tareas y practicas: 20%
- Proyecto: 20%

2.3 Herramientas:

- Eminus
- Entorno controlado para exámenes
- Practical Cryptography in Python - Libro de Referencia

3 Temario

3.1 Descripción

Es un elemento fundamental de la protección de la información.

3.2 Unidad 1

- Cifrado Cesar
- Usos de la criptografía
- Criptografía y criptoanálisis

3.3 Unidad 2

- Funciones de hash
- AES
- Modos de operación de AES
- Padding
- Manejo de llaves y vectores de inicialización

3.4 Unidad 3

- Llaves publicas y privadas
- RSA
- PKCS

3.5 Unidad 4

- MAC, HMAC. CBC-MAC
- Firmas digitales
- Curvas

4 Introducción a la criptografia

- CIA: Confidencialidad -> Impide ver algo a quien no debe verlo. Integridad -> Evitar la manipulación de los datos. Disponibilidad -> Tener la posibilidad de acceder a los datos (que tengamos permiso) en cualquier momento.

Autenticar -> Probar que eres quien dices ser. Autorizar -> Dar acceso al que se autentica mediante permisos.

Secretos

Llaves privadas

Datos pueden estar en 3 estados:

- Reposo -> Esta almacenado en disco.
- Transito -> Estan viajando por la red.
- Uso -> Estan en la memoria

4.1 STRIDE

- STRIDE es un modelo de amenazas, utilizado para ayudar a razonar y encontrar amenazas a un sistema.
- Spoofing: Suplantar la identidad en redes informaticas haciendose pasar por una persona, entidad o sistema de confianza para engañar a la victima. Se puede mitigar por firmas digitales, contraseñas, etc. -> Autenticidad

- Tampering: Alterar, manipular o interferir con algo sin autorización, generalmente con el fin de causar daño, obtener beneficios ilegales o comprometer la integridad de un sistema, proceso o evidencia. Usa la tecnología de hash para mitigarlo. -> Integridad
- Repudiation: El acto de negar, por parte de una persona o sistema, haber realizado una acción, transacción o comunicación específica. -> No repudio
- Information disclosure: Acceso no autorizado o la revelación inadvertida de datos sensibles, confidenciales o privados a personas, sistemas o entidades que no deberían tener acceso a ellos. -> Confidencialidad
- Denegation of service: Hacer que un sistema, servicio o red sea inaccesible para sus usuarios legítimos. -> Disponibilidad
- Elevation of privileges: Otorgar permisos a usuarios que no deben de tener dichos permisos. -> Autorización

4.2 Origen de la criptografía

- Viene del griego kryptos que significa oculto y graphe que significa grafo o escritura.
- Es un ámbito de la criptología que se ocupa de las técnicas de cifrado o codificado destinadas a alterar las representaciones lingüísticas de ciertos mensajes con el fin de hacerlos ininteligibles.
- Se encarga del estudio de los algoritmos, protocolos criptográficos, y sistemas que se utilizan para proteger la información y dotar de seguridad a las comunicaciones y a las entidades que se comunican.
- El criptoanálisis es la parte que se dedica al estudio de los sistemas criptográficos con el fin de encontrar debilidades en los sistemas y romper su seguridad sin el conocimiento de información secreta.

4.3 Criptografía

- Esta en todos lados: ssh, criptomonedas y cualquier aplicación web.
- Es un pilar de la seguridad informática moderna.
- La criptografía no es solo cifrado, se utiliza también para autenticación e integridad de los datos.

- La criptografía bien implementada es efectiva sin importar:
 - Que tan bien educado este el adversario.
 - Que tantas computadoras tenga el adversario.
 - Si el adversario conoce los algoritmos usados.
 - Que tan motivado este el adversario.
- La efectividad de la criptografía depende usualmente del contexto.
- La buena seguridad depende de siempre kde tus elecciones.
- Se debe de tomar la decisión adecuada de acuerdo al contexto del problema.
 - Nunca utilices un metodo criptografico si no entiendes su contexto de uso y su correcta configuración.
 - Nunca utilices un metodo criptografico usando la configuración por defecto a menos que la conozcas.
- Revisa bien los metodos criptograficos que aplicaras muchos tienen vulnerabilidades conocidas.
- TLS: Transport Layer Security, es la base para otras tecnologias como HTTPS y VPN.
- OpenSSL
- Determinismo -> Es predecible

4.3.1 Herramientas

- cryptography: Biblioteca de python con todos los algoritmos criptograficos que se usaran en el curso.
- gmpy2: Biblioteca python para el tratamiento matematico de numeros grandes.

4.3.2 Cifrado Cesar

- Consiste en reemplazar una letra de un mensaje por otra letra, o tambien, desplazar por un valor x (shift-cipher) el numero de letras.
 - Se puede aplica tanto a texto como a binario.
 - * Si es binario debe considerarse el modulo 256
 - * Si es texto debe considerarse el valor ASCII

4.3.3 Tres protecciones principales de la criptografia

- Autenticidad: Firmas digitales, Codigos MAC.
- Confidencialidad: Cifrado, intercambio de llaves.
- Integridad: Hashing.

4.3.4 AES

- Estandar de cifrado asimetrico.
- Advanced Encryption Standard.
- Es el algoritmo de cifrado simetrico mas usado en la actualidad.
- Lo utiliza TLS (HTTPS) y las funciones de cifrado de sistemas de archivos de un OS.
- Debe de saberse utilizar, tiene diferentes modos de operacion, algunos de ellos inseguros.
- Existen variantes, como AES-128 que utiliza una llave de 16 bytes.
- Una llave con contenido aleatorio es una llave valida.

4.3.5 YANAC

- YANAC: You Are Not A Cryptographer. Acronimo popular en la comunidad criptografica.
- Dont roll your own crypto.
- Es mas facil usar mal los metodos criptograficos que usarlos bien.
- Abstente de crear tus propios metodos criptograficos.
- Hasta expertos criptograficos han creado metodos con vulnerabilidades.
- Un nuevo metodo pasa por muchas revisiones de expertos antes de ser usado de forma practica.
- Incluso debes de abstenerte de hacer tus propias

4.3.6 Codificar

- Transformar de una representacion a otra sin perdida.
- Base64

4.3.7 Cifrar

- Es el proceso de convertir informacion sensible en un formato ilegible mediante un secreto (llave) a las que solo partes autorizadas tienen acceso.

4.3.8 Ofuscar

- Se refiere a encubrir el significado haciendola mas confusa y complicada de interpretar.

4.3.9 Hashear

- Es la funcion de aplicar una funcion de hash, es decir, que transforma un conjunto de datos en una cadena alfanumerica unica que actua como una huella digital del contenido original. No es invertible.

4.3.10 Cifrado cesar

- Se basa en reemplazar una letra de un mensaje por otra letra.
- Se puede aplicar tanto a texto como a binario.
- Si es binario debe considerarse el modulo 256, si es texto debe cuidarse el valor ASCII

4.3.11 Aritmetica Modular

- $4 \% 4$
- $(144 + 300) \% 256 = 188$
- $(188 - 300) \% 256 = 144$
- El resultado sera siempre un numero natural.

4.3.12 Binario

- El binario se manipula en paquetes de bytes (8 bits), no se puede manipular directamente menos (al menos en las arquitecturas populares modernas).
- Tarea: Cifrado cesar para archivo de texto.

4.4 Hashing

- $3 \times 2 = 9 \rightarrow$ Biyectivas
- $22 \% 2 = 0 \rightarrow$
- El hashing es uno de los elementos principales de la seguridad criptografía. Involucra el concepto de una función de una sola vía y huella digital.
 - Una función hash solo funciona si cumple con los siguientes requisitos:
 1. Producen valores únicos y repetibles, es decir, son deterministas para cada entrada.
 2. El valor de salida no da ninguna pista (o patrón) con respecto a la entrada que lo produjo.
- Algunas funciones de hash son mejores que otras cumpliendo las propiedades antes mencionadas.
- Buenas: SHA-256, SHA-512
- No tan buenas: MD5, SHA-1. Estas no deberían ser usadas por sus vectores de seguridad.
- Solo funciona con cadenas binarias

4.4.1 Cifrado vs Hashing

- El cifrado permite tener confidencialidad en los mensajes, pero ayuda a verificar la integridad de los mismos.
- Confidencialidad: Solo las personas autorizadas pueden leer el mensaje.
- Integridad: Las personas no autorizadas no pueden cambiar el contenido del mensaje sin que dicho cambio pueda ser percibido.

- Solo porque un mensaje este cifrado (no pueda ser leído) no significa que no pueda ser alterado, incluso de formas que no tengan sentido despues de cifrar.

4.4.2 DIGESTS

- Su traduccion seria “resumen”
- Una funcion de hash produce un resumen o huella digital de algun contenido binario
- Este resumen puede utilizarse como medida para verificacion de tampering (alteraciones de informacion), utilizadas en analisis forense.
- En una transmision la idea es transmitir el mensaje junto con su digest, para verificar que haya llegado sin alteraciones.

4.4.3 hashlib

```
import hashlib md5hasher = hashlib.md5(b'cadenaBin') md5hasher.hexdigest()
md5hasher.update(b'continuacionCadenaBin') md5hasher.hexdigest()
```

4.4.4 Funciones de hash

- Una funcion de hash toma un contenido de cualquier tamano(hasta vacio) y siempre produce un numero de un tamano fijo.
- Por ejemplo, MD5 produce una salida de 16 bytes
- Este tamano tiene una relacion con la seguridad de la funcion entre mas pequenos es peor.
- Dominio -> Codominio
- Una vez mas, las propiedades que una funcion de hash debe de cumplir son:
 - El mismo documento siempre produce el mismo digest.
 - El digest parece aleatorio, si tienes el digest eso no te da pistas del documento de entrada.
 - Idealmente cada documento con contenido diferente deberia tener un digest diferente (lo cual en realidad no es posible).

- Los digests son como huellas digitales: si tienes al sujeto se puede verificar que sea quien dice ser mediante su huella digital.
- Sin embargo, si solo tiene la huella digital no es facil determinar el sujeto que la posee.
- Es facil calcular el digest de un documento, pero si tiene solo el digest es muy dificil determinar el documento que lo produjo, entre mas dificil mejor.
- Por esta razon son llamadas funciones de una sola via, se puede ir de entrada a la salida pero no al revés.

4.4.5 Hashing

- En computacion, el termino funcion de hash casi siempre se refiere a una funcion criptografica.
- Sin embargo, el termino es general
- Por ejemplo: Una funcion hash que establece si un numero es par o impar. Esencialmente, las funciones hash relaciona elementos de un dominio muy grande, con elementos de un codominio relativamente pequeno.
- El tamano del codominio depende de la funcion de hash, por ejemplo, ya se dijo que MD5 genera digest de 16 bytes. Cual es el tamano del codominio para MD5?
- SI se considera un numero muy grande de documentos de entrada es posible que muchos de ellos produzcan el mismo hash.
- Dado lo anterior, las funciones de hash tienen perdida. Se pierde informacion de la fuente al digest.
- Que tenga perdida es importante, de lo contrario habria una forma de regresar del digest al documento original.
- Considerando de nuevo la funcion par
- Cualquier numero puede dar como resultado 0 (par) o 1 (impar)
- Seria posible saber, dado un valor 0 o 1 que numero produjo dicho resultado

- Esta funcion tiene todas las propiedades: tienen perdida, comprime la entrada y es consistente ya que dada la misma entrada siempre produce la misma salida.
- Las propiedades de consistencia, compresion y perdida son comunes en cualquier funcion de hash pero no suficientes para ser seguras o criptograficas.
- Se requieren las sig. propiedades:
 - Resistencia de preimagen.
 - Segunda resistencia de preimagen.
 - Resistencia de colisiones.

1. Resistencia de preimagen

- Una preimagen es el conjunto de entradas de una funcion de hash que producen, en el caso de par o impar, una preimagen son todos los numeros pares que den valor 0.
- Una preimagen de una funcion de hash H y un valor de hash k es el conjunto de valores x por el cual $H(x) = k$
- Al tener un digest, existe un numero infinito de documentos que produjeron dicho digest.
- Eso no quiere decir que dado un digest sea facil encontrar un elemento de la preimagen
- La resistencia de preimagen es basicamente: si tengo un digest y no se como se obtuvo, no pudo ni siquiera encontrar un elemento en la preimagen sin realizar una cantidad ridicula de trabajo computacional.
- Intratable -> Se puede resolver, pero cuesta demasiado
- No computable -> No se puede resolver con una computadora
- Idealmente, los elementos de la preimagen deberian estar muy separados entre ellos y no tener un espaciado predecible
- El hecho de que las funciones de hash tengan perdida ayuda a la resistencia de preimagen
- Si una funcion de hash tiene buena resistencia de preimagen, los ataques en este sentido seran aleatorios o bien consistiran en enumeraciones muy largas (fuerza bruta) hasta encontrar una coincidencia, lo cual no es computacionalmente tratable.

- Al proceso de intentar encontrar un elemento de la preimagen dado un valor de hash se le llama invertir el hash
- Encontrar un inverso debería ser muy difícil.

2. Segunda resistencia de preimagen

- Si tu tienes al menos un documento que produce un hash conocido, sigue siendo muy difícil encontrar otro documento que produzca el mismo hash.
- Esto funciona porque no existe patrón repetible en los elementos de la preimagen.

3. Resistencia a colisiones

- Significa que es difícil encontrar dos entradas que produzcan el mismo hash y garantiza que sea computacionalmente inviable encontrar dos entradas diferentes que produzcan el mismo valor hash.
- No un hash específico sino cualquier hash.
- Es más fácil encontrar una colisión en un hash cualquiera que encontrar un elemento de la preimagen de un hash dado.
- Propiedad avalancha: El valor de hash cambia drásticamente y de forma impredecible, un mínimo del 50% de los bits del hash debería ser diferente.

4.4.6 Seguridad en funciones de hash

- Hash es usado para el almacenamiento de passwords, pero siguen teniendo problemas de seguridad, el más común es la situación donde dos usuarios usen la misma contraseña y por tanto, generen el mismo hash, existen muchas soluciones (A2F, Token físico, etc) pero la más recomendada es agregar un salt.
 - Salt: Es un valor usualmente aleatorio que se debe agregar al password antes de aplicarse a la función de hash. Debido a la propiedad de avalancha, con el salt el hash resultante cambiaría drásticamente. Y como son aleatorios, es poco probable que dos usuarios con la misma password tengan el mismo salt.

`salt = os.urandom(16)`

4.5 Cifrado simetrico

- Cifrado que usa la misma llave para cifrar y descifrar, la llave es un secreto compartido.

4.5.1 AES

- Advanced Encryption Standard
- Lo utiliza TLS (HTTPS) y funciones de cifrado de sistemas de archivos de un OS.
- AES es un cifrador de bloque, cifra por bloques de 16 bytes, si no cumple con los 16 bytes se debe de agregar un padding (datos de relleno).
- Tambien puede cifrar un byte a la vez, esto se conoce como cifrado de flujo y no necesitan padding.
- En general, es mas eficiente cifrar bloques que en flujo.

1. Modos de operacion de AES

(a) ECB - Electronic Code Book

- Cualquier texto plano de 16 bytes tiene 16 bytes correspondientes de salida.
- Cada bloque es tratado independientemente, esa es su debilidad.
 - Puede generar un ataque de texto plano escogido, que se basa en tener muchos mensajes de estructura parecida para enganar con un mensaje falso con una respuesta predecible.
- Parametros: Texto plano y llave.

(b) CBC - Cipher Block Chaining

- Despues de cifrar el bloque aplicar una funcion XOR con el siguiente bloque en texto plano, el resultado se cifra y se guarda ese bloque. Para descifrar se tiene que invertir el proceso.
- Para el primer bloque a cifrar, se utiliza el IV dado que no hay un bloque anterior para hacer XOR, el IV debe de ser del mismo tamaño del bloque (16 bytes).

- Parametros: Texto plano, llave e IV
 - Debe de tener su respectivo padding.
- (c) CTR - Counter Mode
- No requiere de padding
 - Genera un key stream de cualquier longitud a partir de una llave AES de al menos 128 bits (16 bytes), luego aplica XOR para combinarlos.
 - CTR incrementa el contador en uno y asi sucesivamente. Si el texto plano no es multiplo de 16, los ultimos 16 bytes del key stream se truncan para corresponder al texto.
 - Al IV se le llama nonce.
 - CTR puede cifrar de forma independiente.
 - Parametros: key, nonce, texto plano.

4.5.2 Seguridad en cifrado simetrico

- Para tener un cifrado seguro y efectivo se necesita:
 1. Cifrar el mismo mensaje de forma diferente cada vez que se haga, aunque se use la misma llave.
 2. Eliminar patrones predecibles entre los bloques. (Maleabilidad entre bloques)
- Para resolver el primer problema, se utiliza un vector de inicializacion (IV), el cual es una cadena aleatoria externa al texto plano y la llave. El IV es publico ya que solo hace que los mensajes no sean repetibles y no generar patrones.
- Nunca debe reusarse el mismo IV
- Existen ataques donde si conoces el texto plano y el IV (suponiendo que lo reutilizan) puedes obtener el key-stream.

4.6 Cifrado asimetrico

- Las tecnicas asimetricas son variadas, no se limitan solo a cifrado
 - Por ejemplo: firmas digitales e intercambio de llaves
- Algoritmo RSA

- Existen algoritmos derivados de las curvas elipticas

1. Dos llaves

- Se necesitan dos llaves: una publica y otra privada
 - Un mensaje se cifra con la llave publica
 - Ese mensaje solo se puede descifrar con la llave privada
 - La llave publica se comparte y la privada no.

2. RSA

- RSA esta casi obsoleto, debido a multiples vulnerabilidades
- Sirve para ejemplificar el cifrado asimetrico
- Factorización de primos
- Las llaves son basicamente numeros primos muy grandes que se
- Estos numeros son coprimos
- Si solo se tiene uno de los dos

4.6.1 Integridad de mensajes, firmas y certificados

- MACS
- Firmas digitales
- Certificados
- “Keyed Hashes”
- Uso de criptografia asimetrica para integridad y autenticidad de mensajes
- Certificados y su diferencia con las llaves

1. Message Authentication Code (MAC)

- Un cifrado puede ser alterado de formas que tenga sentido.
- Un MAC es cualquier codigo o dato transmitido junto con el mensaje que puede ser evaluado para determinar si el mensaje fue alterado.
- Usualmente se utiliza un hash como

(a) MAC, HMAC, CBC-MAC

- Un MAC seguro no solo es código (hash) también debe incluir una llave.
- Una llave MAC no está relacionada con el cifrado
- Sirve para asegurar que el código de autenticación de mensaje solo puede ser computado por las entidades que conocen la llave.
- No se debe de asumir que el atacante no conoce la función de hash, recuerda que una buena seguridad significa. . .
- MAC protege la integridad del mensaje
- Un atacante sin la llave no debe poder alterar el mensaje sin que se pueda detectar.
- Mientras la llave se mantenga privada, el MAC

i. HMAC

- Hash-based Message Authentication Code
- Es básicamente un hash keyed
- Las funciones de hash son deterministas, dada la misma entrada siempre se produce la misma salida.
- Solo agregar al inicio o al final
- $H(K \text{ XOR } \text{opad}, H((K \text{ XOR } \text{ipad}), \text{text}))$
 - H es la función de hash
 - K es la llave que debe de cumplir ciertos requisitos
 - text es cualquier binario que representa el contenido de entrada
 - opad e ipad son dos valores
 - B es el tamaño del bloque de H (128 bytes)
 - A. Si K es más corto que B, entonces a K se le tienen que agregar ceros de padding hasta ser de longitud B.
 - B. Si K es más largo que B, entonces K se reduce aplicando H a K, o sea $H(K)$
 - C. Si se aplica 2 y la llave queda muy corta, entonces se aplica 1
 - ipad se define con el byte 0x36 repetido B veces
 - opad es el byte 0x5c repetido B veces
 - Los pads parecen arbitrarios pero tienen un sentido de seguridad matemático que ayuda a mitigar

- Primero computas
- ii. CBC-MAC
 - Similar a AES-CBC
 - Recordar que CBC es Cipher Block Chaining
 - Un MAC a menudo se le llama “tag” (mensaje) del mensaje
 - Es info extra que se asocia al principio del contenido
 - En notacion matematica un tag se denota a menudo como t
 - Asi, un MAC sobre un mensaje $m1$ produce un tag $t1$
 - El par $(m1, t1)$ es transmitido al receptor para ser verificado
 - A. MAC Prepend Attack
 - Es posible mandar un mensaje manipulado y hacer que parezca que es el mensaje $M1$ o $M2$, suponiendo que esten generados por la misma llave.
 - B. Proceso de ataque, anteponiendo $M1$ a $M2$
 - Recuperar el primer bloque de $M2$, denominado $M21$ (como es AES se toman los primeros 16 bytes)
 - Recuperar el resto de bloques de $M2$, denominado $M2r$
 - Obtener $M'2 = t1 \text{ XOR } M21$
 - $M3 = M1 + M'2 + M2r$ (+ significa concatenacion)
 - Con esto $t3 = t2$
 - Basicamente lo que hace el atacante es reemplazar el primer bloque de $M2$ por uno manipulado.
 - La victima vera todo el mensaje $M1$ completo (con su padding), un bloque que se vera posiblemente corrupto y el mensaje $M2$ sin su primer bloque.

2. Firmas digitales

- Protocolo reto-respuesta
 - (a) El cliente recibe la llave publica del servidor (certificado).
 - (b) El cliente le manda un reto al servidor.
 - (c) El cliente le pide al servidor que firme el dato.
 - (d) El servidor firma el dato y regresa la firma.
 - (e) El cliente comprueba la firma con la llave publica.